API 接口使用手册

> **版本：** v1.0.6
>
> **更新时间：** 2026-09-11

版本更新

序号	版本	更新内容	日期
1	v1.0.0	初始版本	2026-06-09
2	v1.0.1	增加硬件准备步骤；优化状态查询接口；完善日志记录与测试接口	2026-06-09
3	v1.0.2	优化夹爪创建接口和接口描述	2026-06-10
4	v1.0.3	适配固件单位变更	2026-07-15
5	v1.0.4	添加清错、急停和软件复位接口	2026-07-24
6	v1.0.5	添加自动标零和夹持接口；力矩环统一为额定力矩百分比	2026-07-28
7	v1.0.6	适配 S-Module-R35 RS485；改用完整驱动配置创建夹爪； 按功能与使用方法整理接口说明	2026-09-11

目录

- 概述
- 硬件准备
- 快速开始
- 创建与配置
- 生命周期与标零
- 保护与故障管理
- 位置控制
- 速度控制
- 力与电机电流控制

- 夹持控制
- 控制模式
- 状态查询
- 测试执行
- 日志控制
- 接口单位约定

概述

`GripperController` 是面向用户的夹爪控制类，提供初始化、标零、运动控制、状态查询和日志记录等接口。

用户须知

本说明书中提供的信息包含一个或多个产品的一般使用说明。遵循本说明书进行安装和使用不能替代特定应用的测试和风险评估。公司不对因误用或误解本使用说明书中所述任何信息而产生的后果负责。
如果任何说明被认为描述不清楚，用户有责任在使用前联系公司以澄清说明不清楚的问题。
如果用户发现此说明书有任何错误的地方或对此有改进的建议，请联系并通知公司对应负责人员。

硬件准备

1. 断电完成电机、编码器、电源、限位/急停和 RS485 接线，确保夹爪运动范围内无人员或异物。
2. 将 R35 通信接口的 RS485+、RS485-、GND 分别连接到 USB-RS485 转换器对应端子。
3. 给每台驱动器设置唯一的 Modbus 地址，允许范围为 `1~127`；地址 `0` 不可用于创建驱动实例。
4. 将程序波特率设置为驱动器当前实际波特率。
5. 连接转换器后确认串口设备存在，例如：

   ```
   ls -l /dev/ttyUSB0
   ```

6. 确认当前用户具有串口读写权限，并关闭所有占用该串口的调试工具；同一从站只运行一个控制进程。

> **标零要求：** 第一次通电使用位置或速度控制前，必须在安全条件下调用 `initialize_zero()`。该接口会主动张开夹爪寻找机械极限，完成初始标零操作。

快速开始

```cpp
#include "gripper.h"

#include <chrono>
#include <iostream>
#include <thread>

int main() {
    MotorDriverConfig config;
    config.motor_id = 1;
    config.protocol_type = "";            // 空值默认使用 smodule_modbus
    config.bus_type = "";                 // 空值默认使用 Rs485
    config.bus_interface = "/dev/ttyUSB0";
    config.motor_type = "smodule_r35";
    config.motor_direction = 1;
    config.master_id_offset = 0;
    config.motor_zero_offset = 0.0;
    config.baud_rate = 115200;             // 必须与驱动器实际设置一致

    auto gripper = GripperController::create(config);
    if (!gripper || gripper->init() != 0) {
        return 1;
    }

    try {
        gripper->clear_error();

        // 首次通电时执行。标零成功后电机处于失能状态。
        if (!gripper->initialize_zero()) {
            gripper->deinit();
            return 1;
        }

        // 设置位置模式并使能。
        gripper->set_control_mode(MotorDriver::POS);
        gripper->enable();
        std::this_thread::sleep_for(std::chrono::milliseconds(500));

        // R35 位置接口必须传入大于 0 的速度。
        gripper->set_gripper_distance(0.06, 0.01f);

        gripper->disable();
        gripper->deinit();
    } catch (const std::exception &e) {
        std::cerr << e.what() << '\n';
        gripper->disable();
```

```
        gripper->deinit();
        return 1;
    }
}
```

创建与配置

`create(const MotorDriverConfig &config)`

根据 `config` 创建夹爪控制器。调用前必须填写完整、有效并与现场设备一致的配置。
创建成功返回控制器实例，失败返回 `nullptr` 。

字段	类型	说明
`motor_id`	`uint16_t`	Modbus 从站地址，范围 `1~127`
`protocol_type`	`std::string`	留空时使用 `"smodule_modbus"` ；也可显式填写该值
`bus_type`	`std::string`	留空时使用 `"Rs485"` ；也可显式填写该值
`bus_interface`	`std::string`	串口设备路径，例如 `"/dev/ttyUSB0"`
`motor_type`	`std::string`	必须填写 `"smodule_r35"`
`motor_direction`	`int`	逻辑方向，只允许 `1` 或 `-1` ；本夹爪标定使用 `1`
`master_id_offset`	`uint16_t`	当前忽略该字段，填写 `0`
`motor_zero_offset`	`double`	软件零偏，单位 rad；不使用时填写 `0.0`
`baud_rate`	`int`	串口波特率，使用 `115200`

生命周期与标零

方法	功能	返回值/结束状态
`init()`	初始化夹爪及通信	`0` 成功， `1` 失败
`deinit()`	结束夹爪控制并释放通信资源	`void` ；结束后未初始化且失能
`enable()`	使能电机	`void`
`disable()`	失能电机	`void`
`set_zero()`	将当前位置设为零位	成功返回 `true` ；结束后失能
`initialize_zero()`	自动寻找张开端机械零位并标零	成功返回 `true` ；结束后失能

`initialize_zero()`

自动张开夹爪至机械极限并建立位置零点。首次通电或零位失效后，在夹爪空载、运动空间安全时调用。
成功返回 `true` ，失败返回 `false` ；调用结束后电机处于失能状态。

```
if (!gripper->initialize_zero()) {
    // 标零失败，停止后续运动。
}

gripper->set_control_mode(MotorDriver::POS);
gripper->enable();
```

`set_zero()`

将夹爪当前位置设为零位。仅在夹爪已经位于确定的机械零位且完全停止时调用；成功返回 `true` ，失败返回 `false` 。调用结束后如需运动，应重新设置控制模式并使能。

保护与故障管理

`clear_error()`

清除驱动器当前错误。调用后使用 `get_error_id()` 检查结果；如果故障源没有消除，错误可能再次出现。

```
gripper->clear_error();
if (gripper->get_error_id() != 0) {
    // 故障仍存在，禁止继续运动。
}
```

位置控制

调用顺序为： `set_control_mode(MotorDriver::POS)` → `enable()` → 位置指令。
位置控制依赖有效零位，因此首次使用前必须完成标零。

`set_gripper_distance(double distance_m, float speed_mps = 0.0f)`

控制夹爪移动到指定开度。控制时必须显式传入正速度。

参数	类型	单位	说明
`distance_m`	`double`	m	两指软垫之间的目标距离；当前 10 mm 软垫配置下有效范围为 `[0, 0.085]`
`speed_mps`	`float`	m/s	轨迹速度幅值；要求有限且大于 `0`

返回值	单位	说明
`>= 0`	deg	对应的电机目标角度
`-1.0`	—	`distance_m` 超出有效范围，未下发命令

调用前先设置 `POS` 模式并使能。检查返回值是否为 `-1.0` ，并捕获通信或参数异常。

```
double target_deg = gripper->set_gripper_distance(0.06, 0.01f);
if (target_deg < 0.0) {
    // 目标距离无效。
}
```

get_gripper_distance()

获取当前两指软垫之间的距离。用于显示位置、检查到位情况或记录数据。

返回值	单位	说明
`double`	m	当前两指软垫之间的距离

速度控制

调用顺序为： `set_control_mode(MotorDriver::SPD)` → `enable()` → 速度指令。

set_gripper_velocity(float speed)

控制夹爪以指定速度持续张开或闭合。调用前先设置 `SPD` 模式并使能；停止时下发 `0` 。

参数	类型	单位	说明
`speed`	`float`	m/s	夹爪距离变化率；正值增大距离（张开），负值减小距离（闭合）， `0` 停止

```
gripper->set_gripper_velocity(-0.01f);  // 闭合
gripper->set_gripper_velocity(0.0f);    // 停止
```

get_gripper_velocity()

获取当前夹爪开合速度，用于状态显示或运动监测。

返回值	单位	说明
`float`	m/s	当前夹爪距离变化率

力与电机电流控制

夹爪末端力接口

`set_gripper_torque(float force_n)`

按目标夹爪末端力控制夹持输出。名称沿用历史 API，但参数单位是 N。调用前先设置 `CUR` 模式并使能；需要撤销输出时传入 `0`。

参数	类型	单位	说明
`force_n`	`float`	N	目标夹爪末端力；正值为闭合方向，负值为张开方向，`0` 撤销输出

`get_gripper_torque()`

获取当前估算的夹爪末端力，用于状态显示和夹持监测。

返回值	单位	说明
`float`	N	估算的当前夹爪末端力

> 力值由模型估算，并非力传感器测量结果。额定力矩、丝杠效率、摩擦和机构参数存在误差，对力精度有要求时必须使用外部力传感器标定。

夹持控制

`grip(float approach_torque_percent, float holding_torque_percent)`

用较小输出接近物体，接触后增加到设定保持输出并持续夹持。调用前先设置 `CUR` 模式并使能。

参数	类型	单位	说明
`approach_torque_percent`	`float`	%	接近阶段电流百分比，必须满足 `0 < approach <= holding`
`holding_torque_percent`	`float`	%	最终保持电流百分比，最大 `180%`

返回值	说明
`true`	完成夹持并保持最终输出
`false`	参数无效、电机未使能或未能完成夹持

```
gripper->set_control_mode(MotorDriver::CUR);
gripper->enable();

if (gripper->grip(15.0f, 30.0f)) {
    // 最终输出会持续保持，完成操作后由调用方显式撤销。
    gripper->motor_torque_cmd(0.0f);
}
```

`grip()` 是阻塞接口，并且不会自动切换控制模式。

控制模式

`set_control_mode(uint8_t mode)`

设置夹爪控制模式。切换模式后必须重新调用 `enable()`，再下发对应类型的控制指令。

枚举名	功能
`MotorDriver::POS`	位置控制
`MotorDriver::SPD`	速度控制
`MotorDriver::CUR`	力/电流控制

`get_control_mode()`

获取当前选择的控制模式，返回值对应上表中的公共模式值。

状态查询

方法	返回类型	说明
`refresh_state()`	`bool`	刷新电机状态；成功返回 `true`，失败返回 `false`
`inited()`	`bool`	查询是否已初始化
`enabled()`	`bool`	查询是否已使能
`disabled()`	`bool`	查询是否已失能
`get_error_id()`	`uint32_t`	获取错误位图；`0` 表示当前未报告错误
`wait_until_stopped(...)`	`bool`	在限定时间内等待夹爪停止；停止返回 `true`，超时返回 `false`

```cpp
if (gripper->refresh_state()) {
    uint32_t error_id = gripper->get_error_id();
}
```

wait_until_stopped(...)

```cpp
bool wait_until_stopped(
    int timeout_ms = 10000,
    int sample_interval_ms = 100,
    float position_threshold_deg = 0.3f,
    int stable_sample_count = 3);
```

等待夹爪停止，适用于位置到位、自动标零和接触检测。

参数	单位	说明
timeout_ms	ms	最大等待时间
sample_interval_ms	ms	状态检查间隔
position_threshold_deg	deg	允许的位置变化阈值
stable_sample_count	次	判定停止所需的连续稳定次数

参数无效或等待超时返回 `false`。返回 `true` 只表示夹爪已经停止，不能区分正常到位、接触物体或机械卡滞。

日志控制

开启与关闭

```cpp
gripper->start_pos_log();     // log/pos_log_YYYYMMDD_HHMMSS.csv
gripper->start_spd_log();     // log/spd_log_YYYYMMDD_HHMMSS.csv
gripper->start_torque_log();  // log/torque_log_YYYYMMDD_HHMMSS.csv
gripper->stop_log();
```

同一时刻只开启一种日志，结束记录后调用 `stop_log()`。文件保存在程序运行目录下的 `log/`。

记录状态

方法	说明
log_pos_state(double target_m)	记录目标距离、目标角度、实际距离、实际角度、误差和到位结果

方法	说明
`log_spd_state()`	记录命令/反馈速度、当前位置和经过时间
`log_torque_state(float cmd_force_n)`	记录命令/反馈夹爪末端力、电机位置、速度和经过时间

使用 `run_velocity_test()` 或 `run_torque_test()` 时无需另外调用状态记录接口；未开启日志时只输出终端信息，不写入 CSV。

CSV 字段

日志	字段
位置	`time,set_m,cmd_deg,actual_m,actual_deg,err_m,result`
速度	`time,cmd_mps,cmd_motor_rpm,actual_motor_rpm,actual_mps,pos_m,elapsed_s`
夹爪末端力	`time,cmd_force_n,actual_force_n,pos_deg,spd_rpm,elapsed_s`